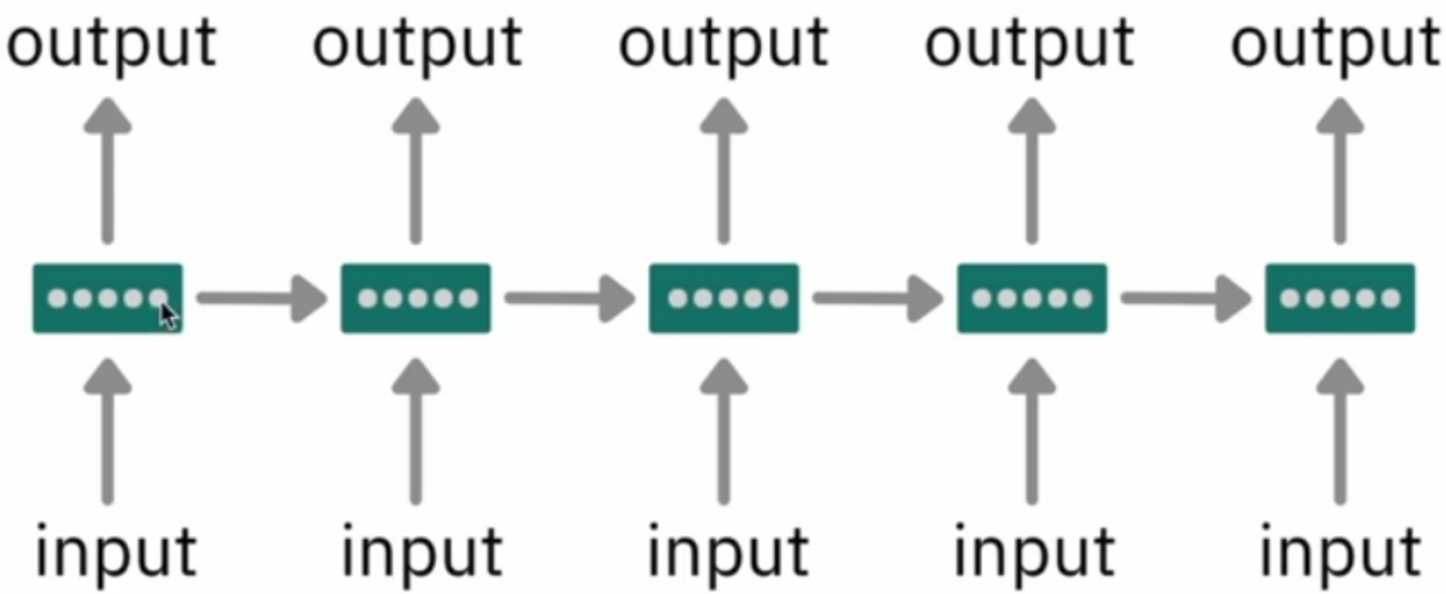
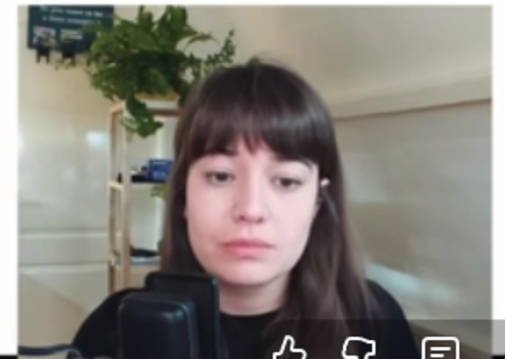
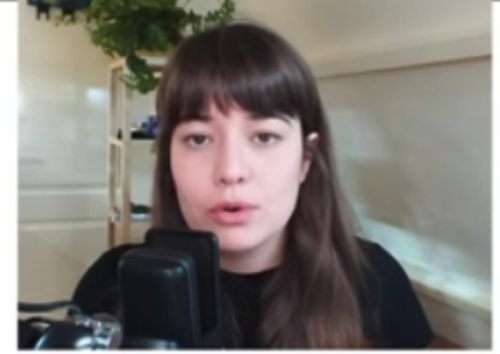
RNNs are recurrent







Types of using RNNs



- Sequence to sequence (price forecasting)
- Sequence to vector (scam or not)
- Vector to sequence (image captioning)
- Encoder-decoder (translation)

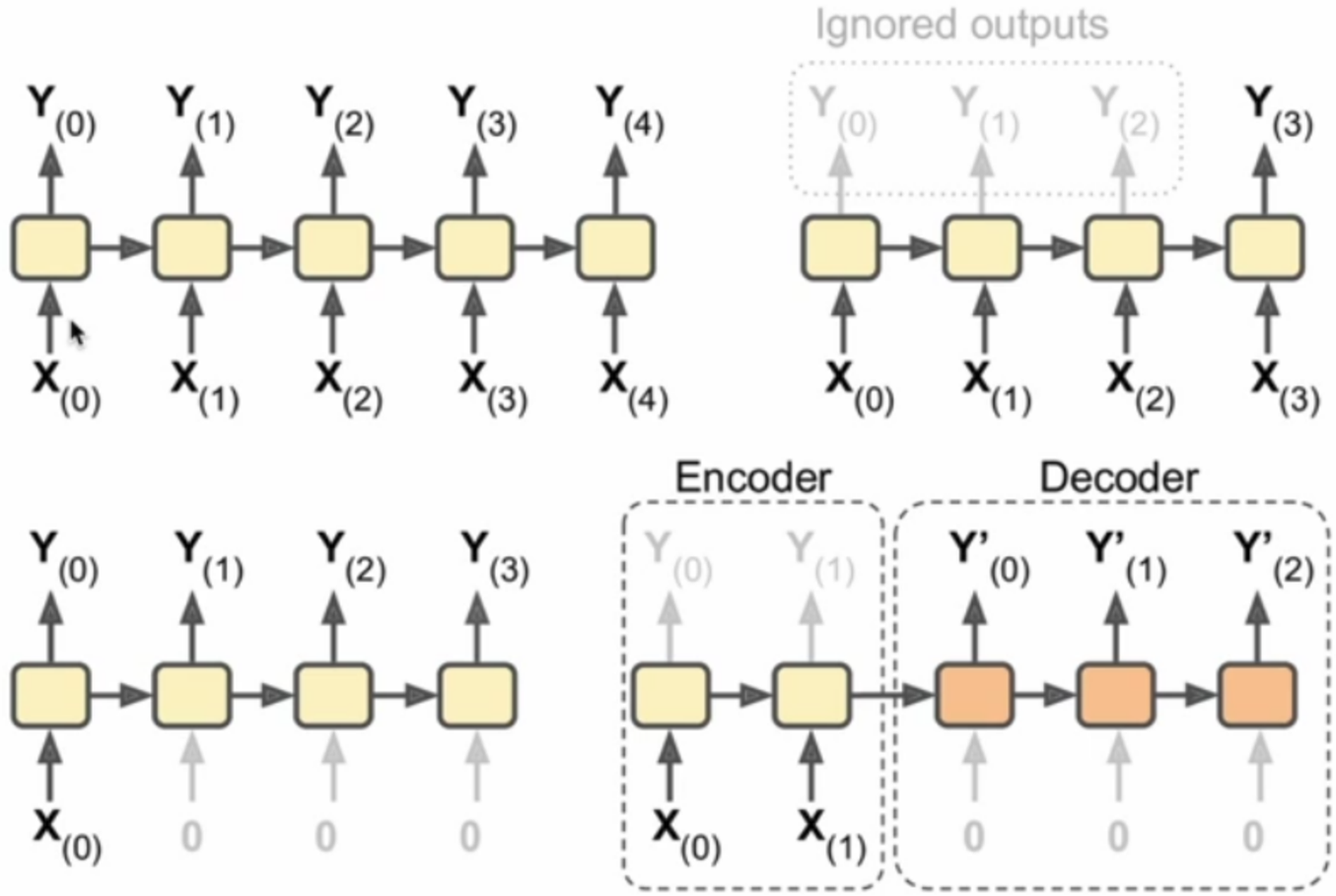
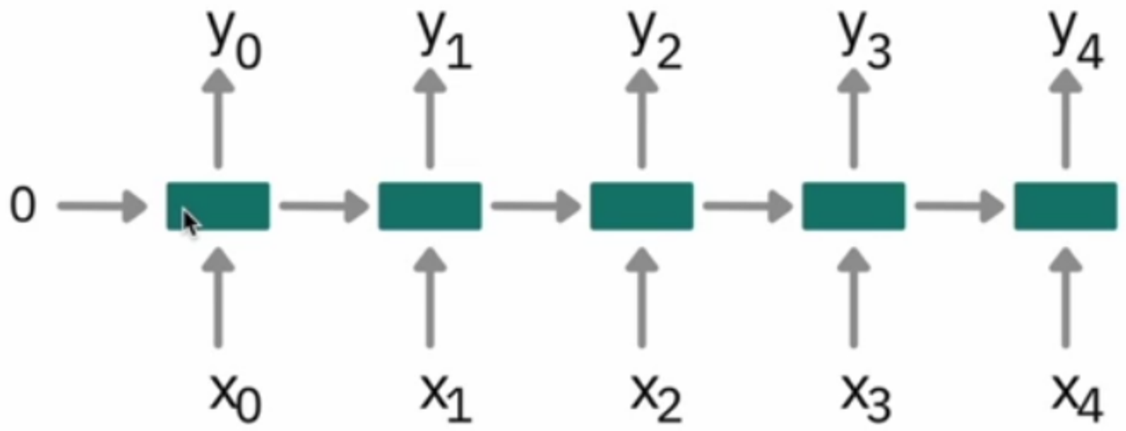
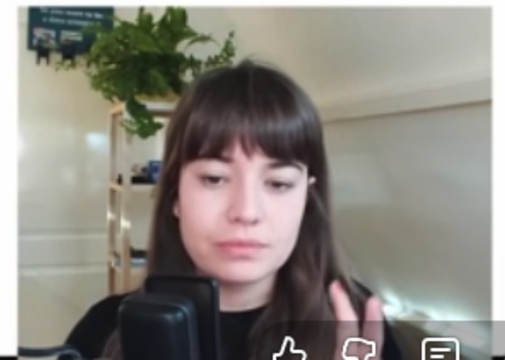


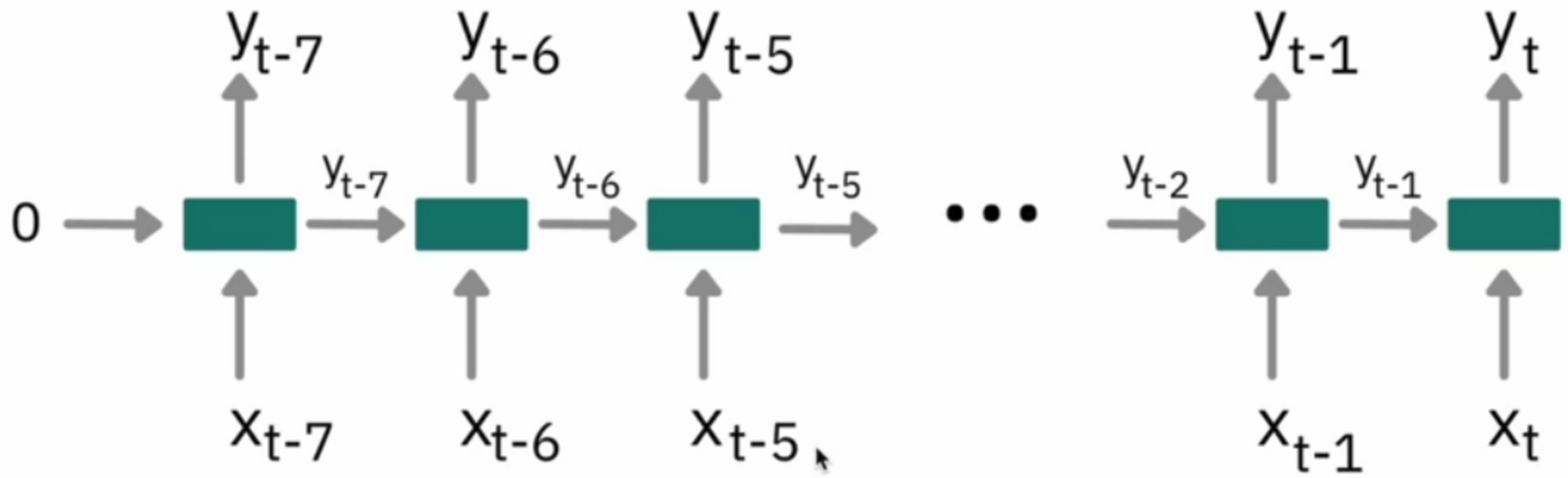
Image from Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow by Aurélien Géron.

Backpropagation through time

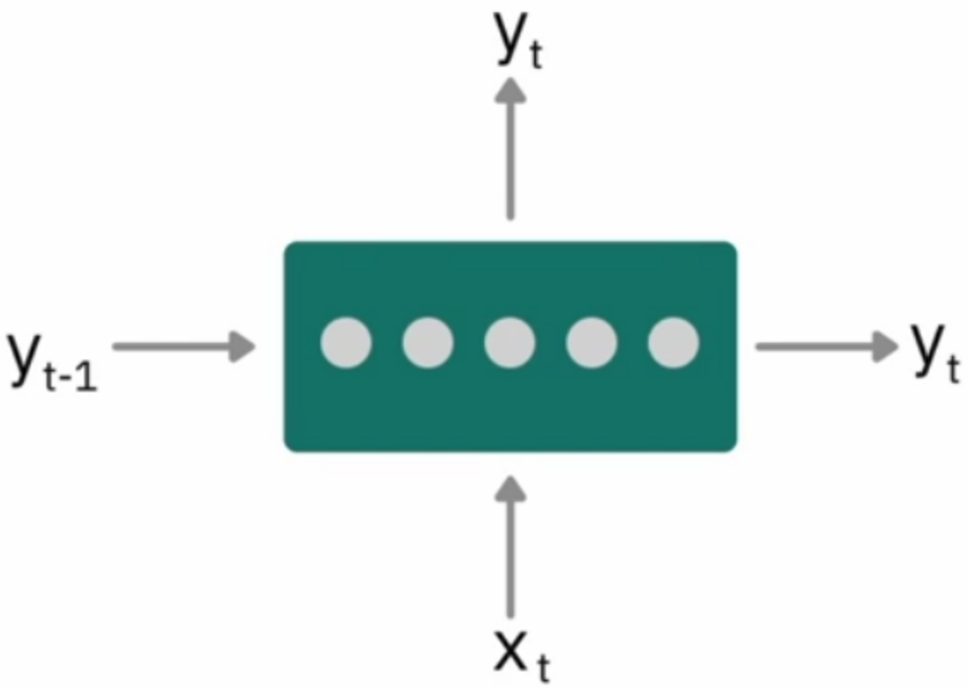
- Network output is calculated in its unrolled form
- Cost is calculated
- And then gradients are calculated
- Gradients are passed back and weights are updated
- Weight matrixes are all the same since it's actually only 1 cell

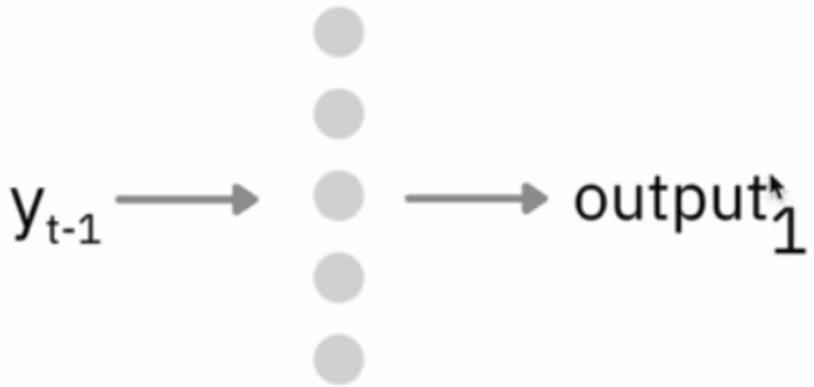


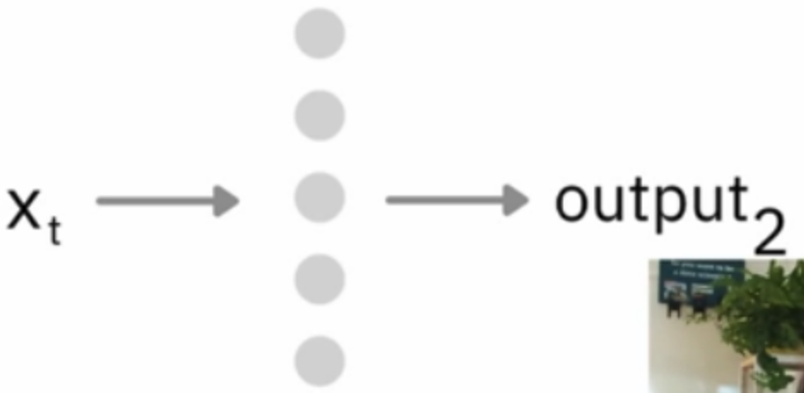



y_{t-7}
y_{t-6}
y_{t-5}
y_{t-1}
y_t
0
y_{t-7}
y_{t-6}
y_{t-5}
y_{t-2}
y_{t-1}
x_{t-7}
x_{t-6}
x_{t-5}
x_{t-1}
x_t

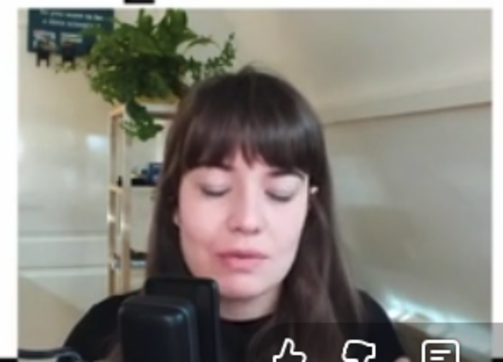
How is the output calculated?







$$a\left(output_1 + output_2 + bias\right)$$



$$y_t = \alpha\left(W_x x_t + W_y y_{t-1} + b\right)$$

$$W_x = \begin{bmatrix} 0.7 & 0.1 \\ 0.3 & 0.9 \\ 0.8 & 0.6 \end{bmatrix}$$

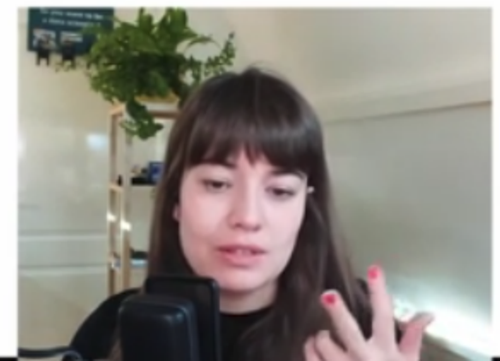
neurons

timesteps

$$x_t = [0.7 \quad 0.3 \quad 0.6]$$

timesteps

$$\begin{bmatrix} 0.7 & 0.1 \\ 0.3 & 0.9 \\ 0.8 & 0.6 \end{bmatrix} [0.7 \quad 0.3 \quad 0.6] = [0.49 \quad] + 0.09$$



$$y_t = \alpha\left(W_x x_t + W_y y_{t-1} + b\right)$$

$$W_x = \begin{bmatrix} 0.7 & 0.1 \\ 0.3 & 0.9 \\ 0.8 & 0.6 \end{bmatrix}$$

neurons

timesteps

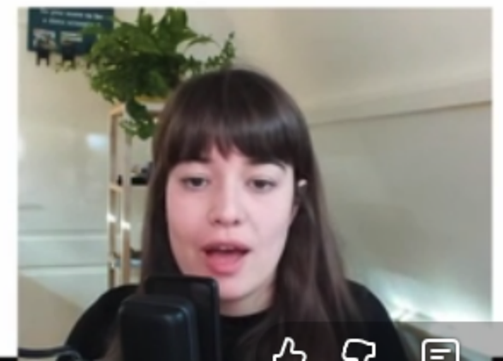
$$x_t = [0.7 \quad 0.3 \quad 0.6]$$

timesteps

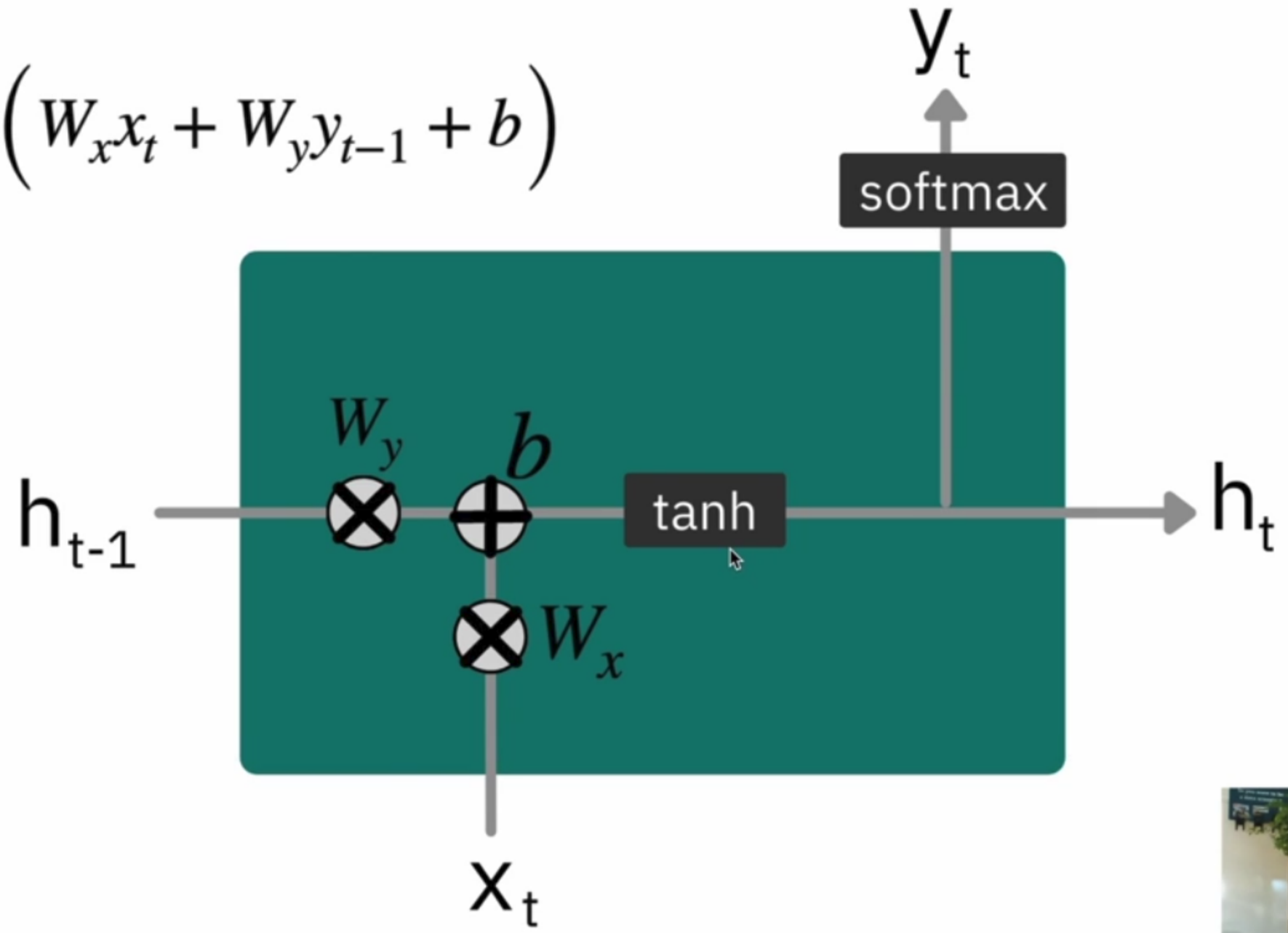
$$\begin{bmatrix} 0.7 & 0.1 \\ 0.3 & 0.9 \\ 0.8 & 0.6 \end{bmatrix} [0.7 \quad 0.3 \quad 0.6] = [0.49 \quad 0.07] = [1.06 \quad 0.70]$$

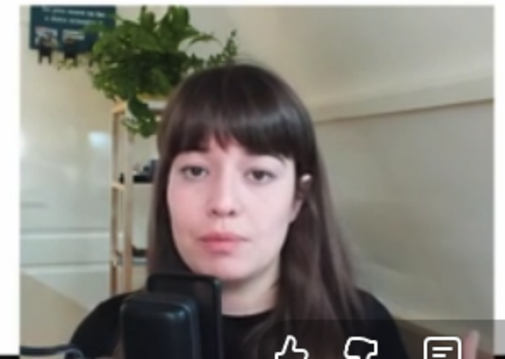
$$\begin{matrix} + & + \\ 0.09 & 0.27 \\ + & + \\ 0.48 & 0.36 \\ = & = \\ 1.06 & 0.70 \end{matrix}$$

neurons

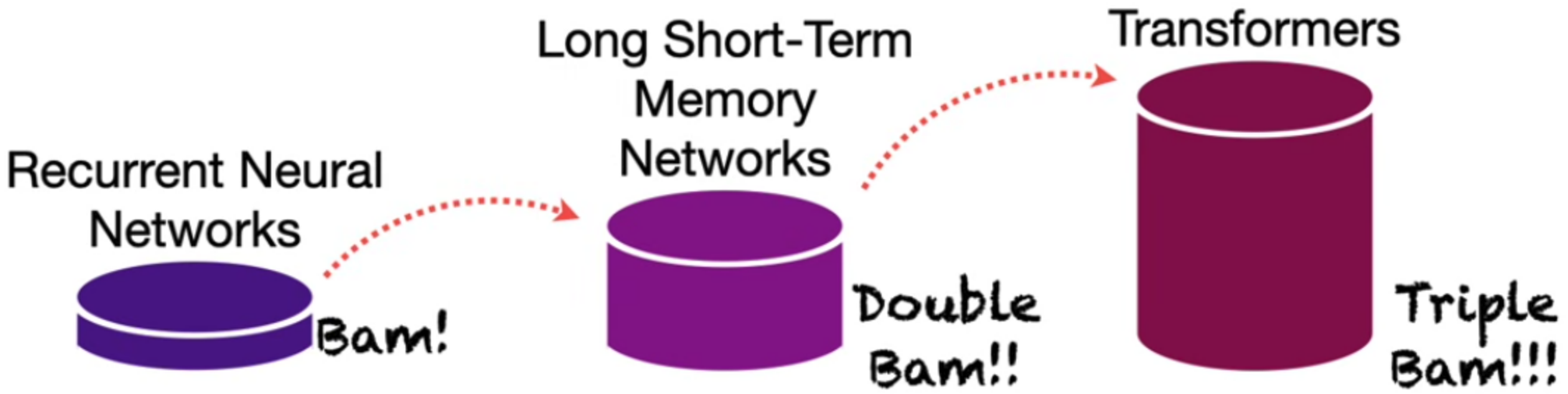


$$y_t = \alpha\left(W_x x_t + W_y y_{t-1} + b\right)$$

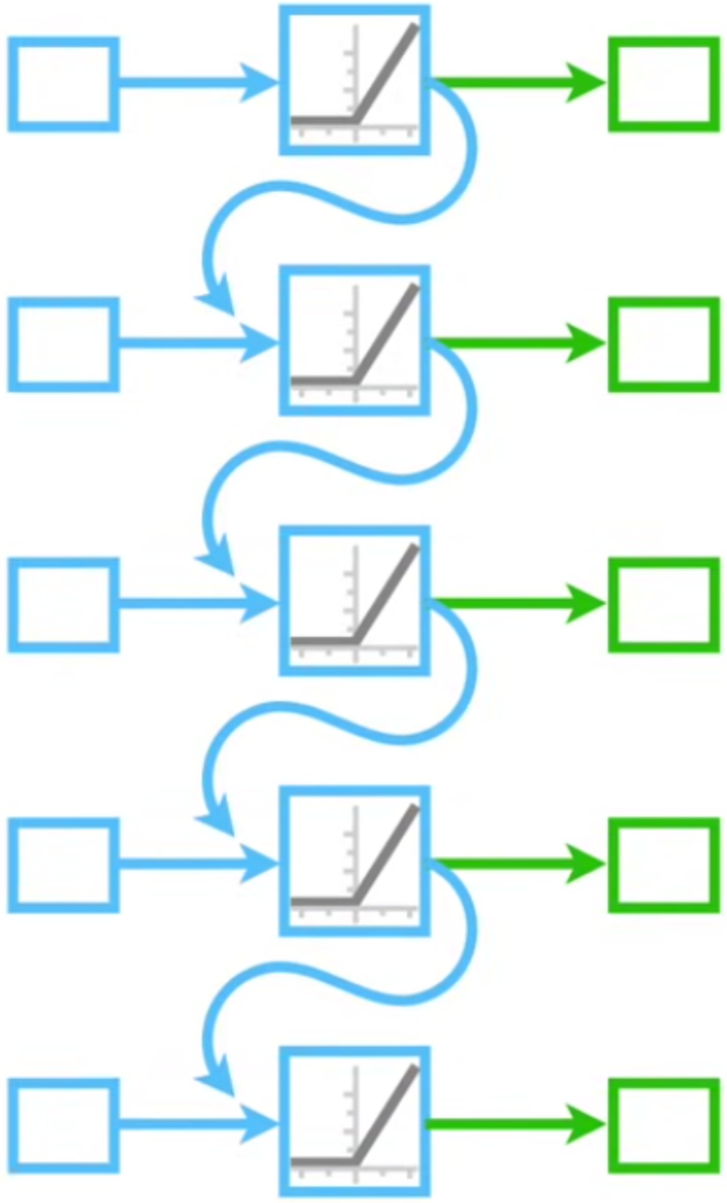




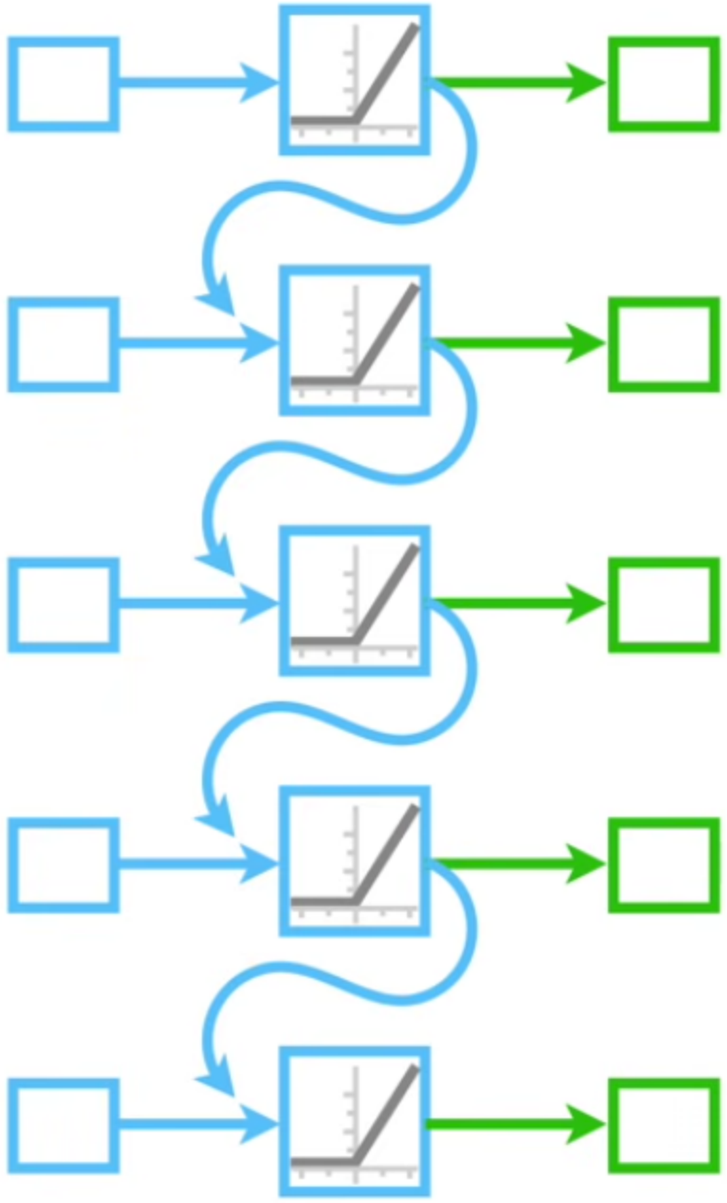
ALSO NOTE: Although ***basic***, or ***vanilla*** **Recurrent Neural Networks** are awesome, they are usually thought of as a stepping stone to understanding ***fancier*** things like **Long Short-Term Memory Networks** and **Transformers**, which we will talk about in future **StatQuests**.



One big problem is that the more we **unroll** a recurrent neural network, the harder it is to train.



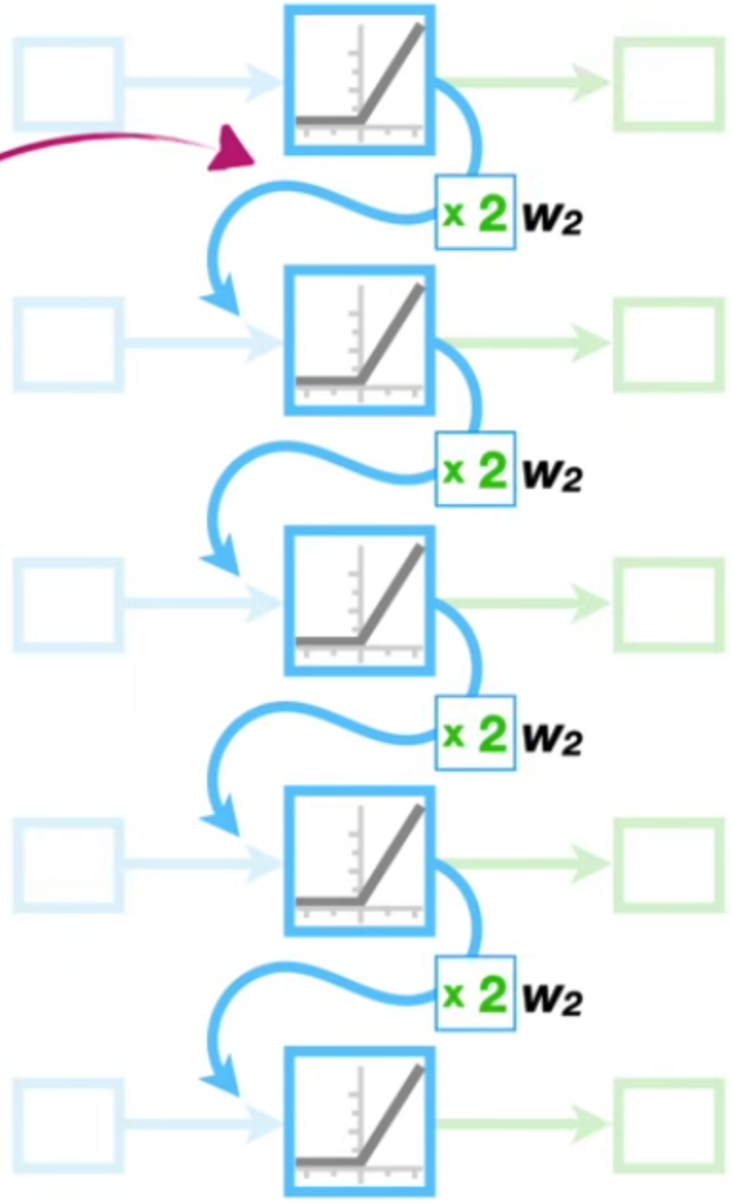
This problem is called **The Vanishing/Exploding Gradient Problem.**



In our example, the gradient will ***explode***, when we set w_2 to any value larger than **1**.



So let's set
$w_2 = 2$.





And that means the first input value is amplified **16** times before it gets to the final copy of the network.

$$\textbf{Input}_1 \times 2 \times 2 \times 2 \times 2$$

$$= \textbf{Input}_1 \times 2^4 = \textbf{Input}_1 \times 16$$

$$= \textbf{Input}_1 \times w_2^{\text{Num. Unroll}}$$



Now, if we had **50** sequential days of stock market data, which, to be honest, really isn't that much data…

= **Input**$_1$ x 2^{50}

= **Input**$_1$ x $w_2^{\text{Num. Unroll}}$



And this **HUGE NUMBER** is why they call this an **Exploding Gradient Problem.**

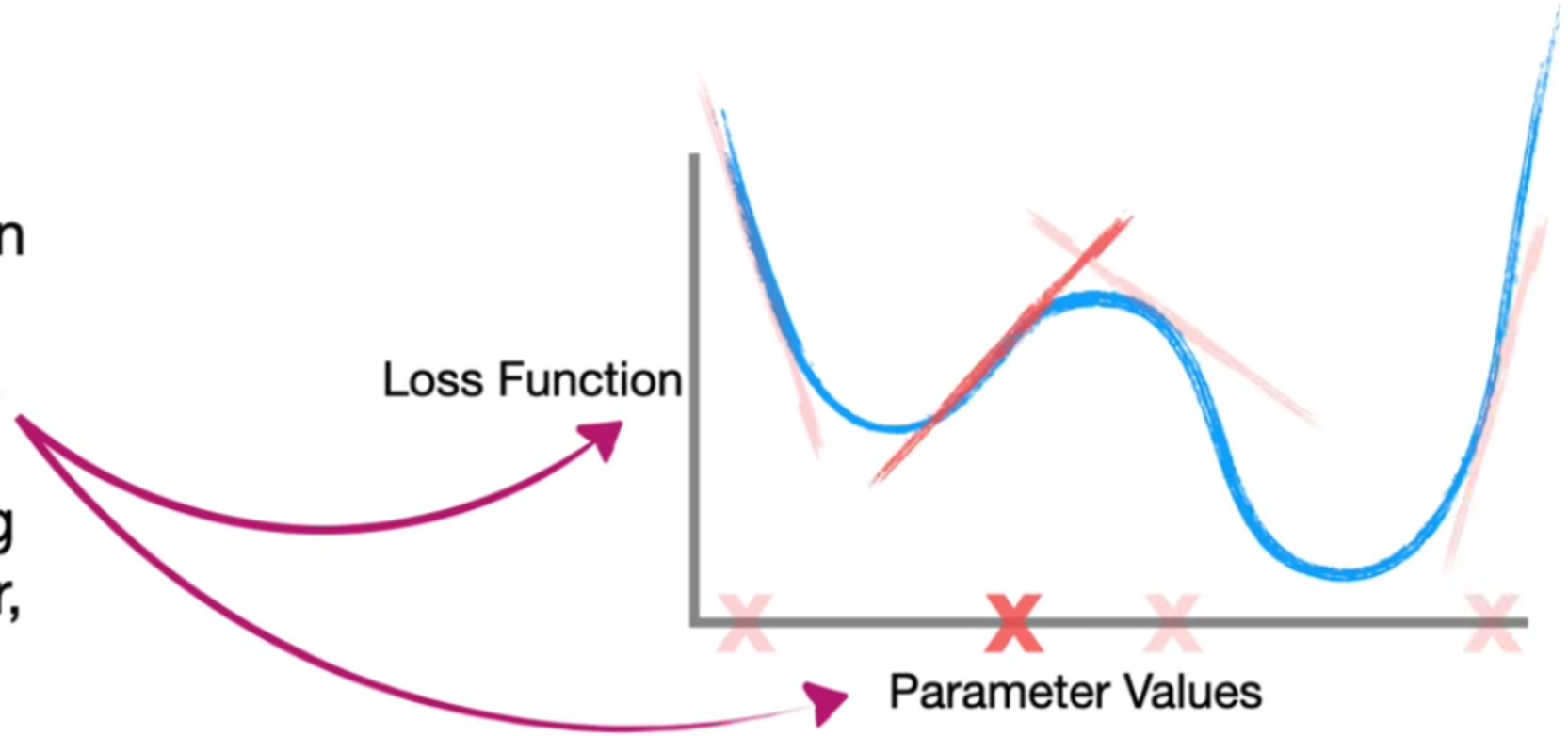
= **Input**$_1$ x 2^{50} = **Input**$_1$ x A HUGE NUMBER

= **Input**$_1$ x $w_2^{\text{Num. Unroll}}$

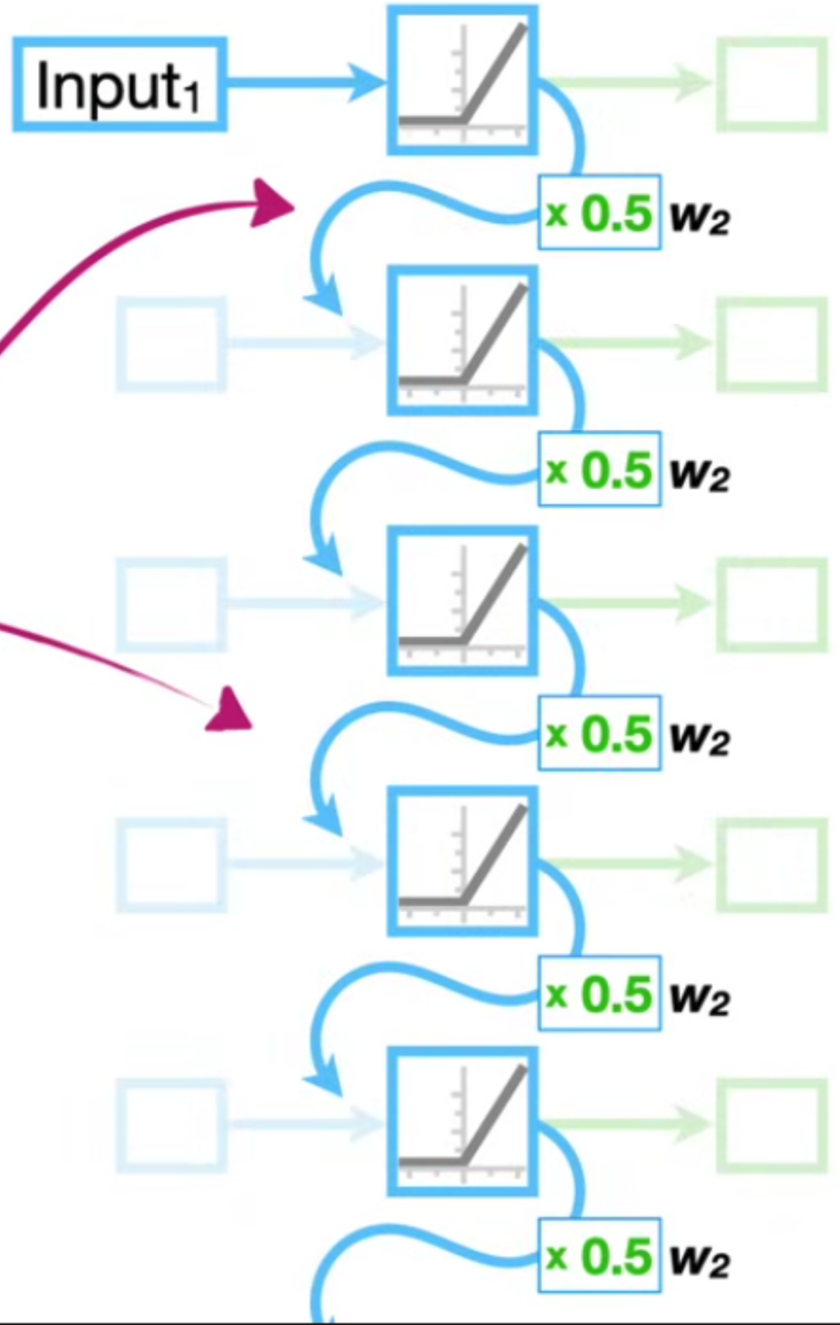


However, when the **Gradient** contains **A HUGE NUMBER**, then we'll end up taking relatively large steps.

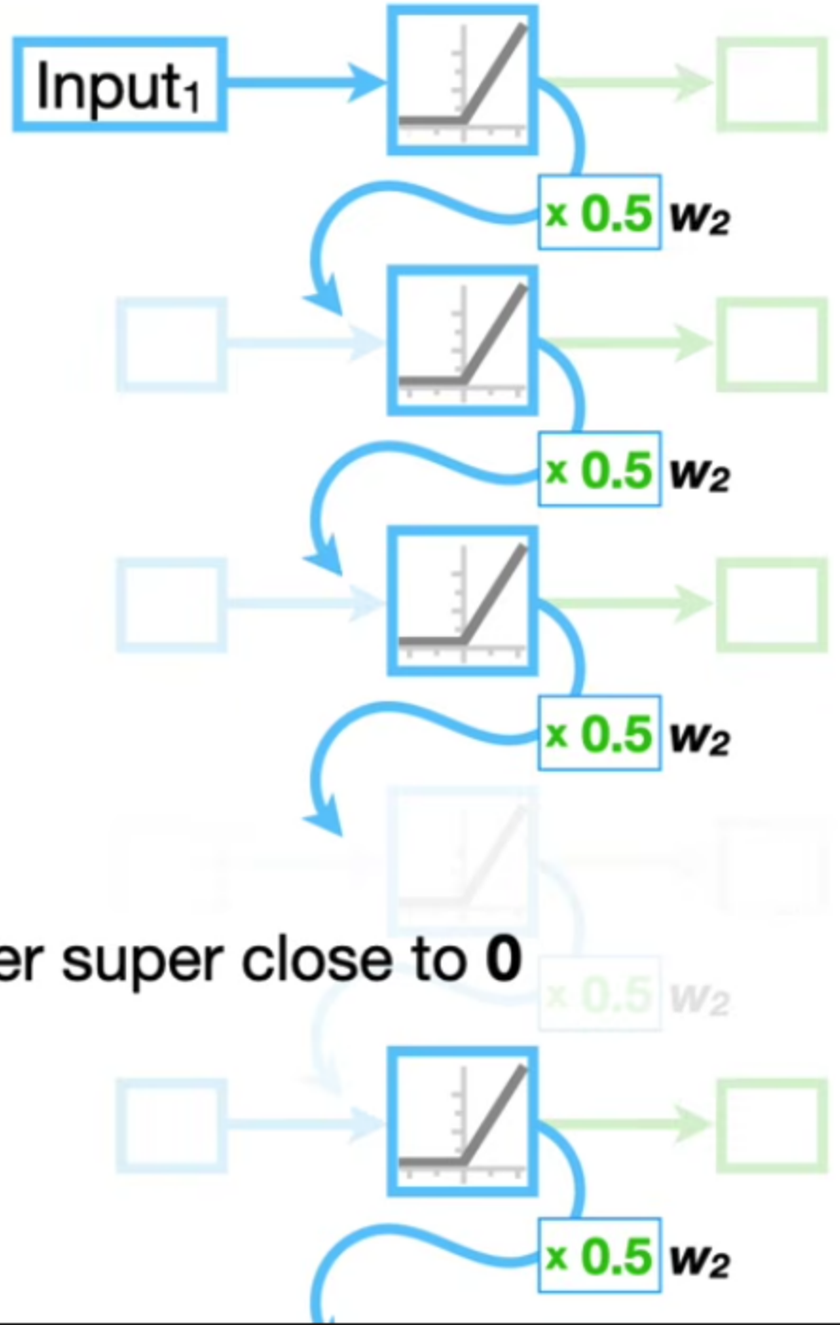
And instead of finding the optimal parameter, we'll just bounce around a lot.



To illustrate **The Vanishing Gradient Problem**, let's set $w_2 = 0.5$.



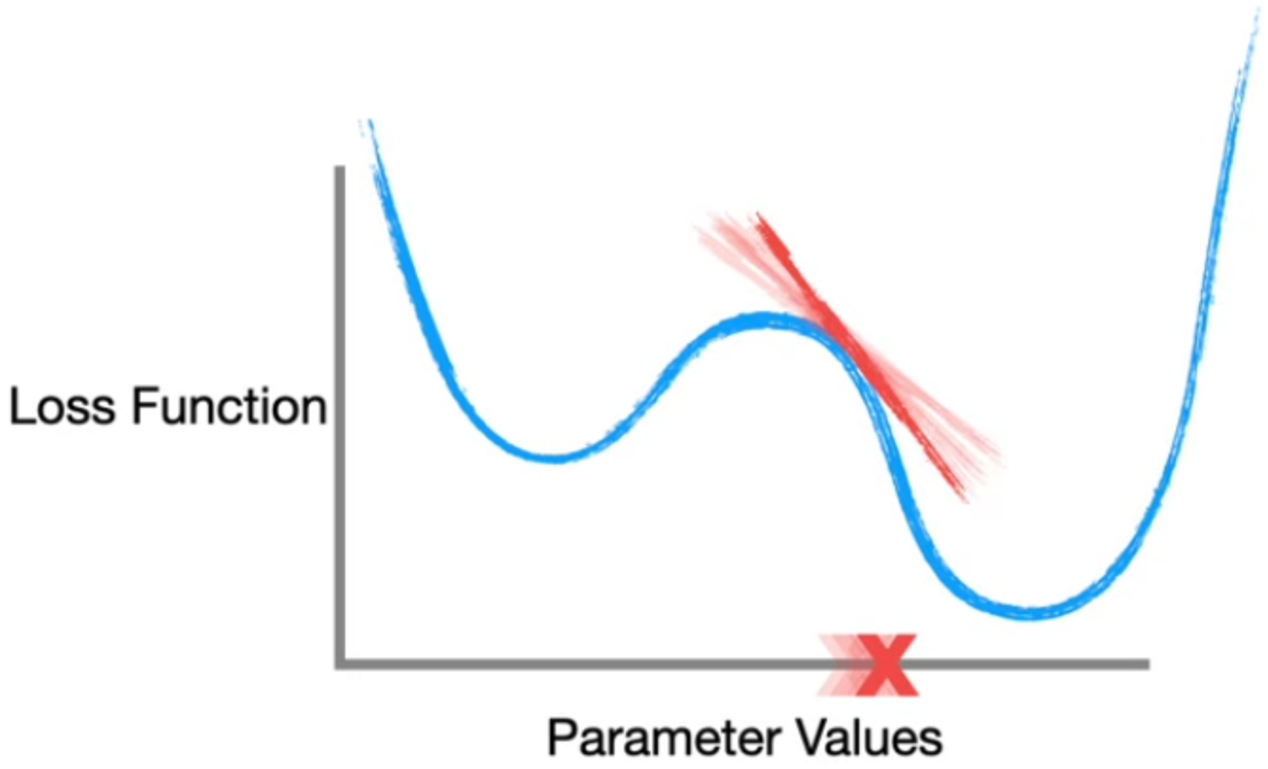
Because this number is **super close to 0**, this is called the **Vanishing Gradient Problem**.



Input$_1$ × 0.5^{50} = **Input**$_1$ × a number super close to **0**

Input$_1$ × $w_2^{\text{Num. Unroll}}$

And as a result, we end up hitting the maximum number of steps we are allowed to take before we find the optimal value.





YES! And we'll talk about a popular solution called **Long Short-Term Memory Networks** in the next **StatQuest!!!**